

Задача А

Условие: "Юный сетевик"

Юный сетевой администратор Игорь только что устроился на работу в крупную IT-компанию "МегаБайт". В первый же день начальник поручил ему важную задачу: проверить корректность настройки сетевой инфраструктуры компании.

В компании есть несколько отделов, и каждому выделена своя **подсеть** для организации внутренней работы. Однако предыдущий администратор допустил ошибку в конфигурации, и теперь некоторые подсети могут **пересекаться**, что приводит к конфликтам в сети и сбоям в работе.

Игорю нужно срочно проверить, есть ли среди выделенных подсетей такие, которые имеют общие IP-адреса. Помогите ему справиться с этой задачей!

Определения

IPv4-адрес — это уникальный идентификатор устройства в сети, состоящий из 4 чисел от 0 до 255, разделённых точками. Например: 192.168.1.1

В двоичном представлении IPv4-адрес — это 32-битное число.

Подсеть — это диапазон IP-адресов, который задаётся в формате IP-адрес/маска, где:

- **IP-адрес** — базовый адрес подсети
- **маска** — число от 0 до 32, показывающее количество фиксированных битов в начале адреса

Например, запись 192.168.1.0/24 означает подсеть, в которой первые 24 бита фиксированы, а последние 8 битов могут изменяться. Эта подсеть содержит $2^{32-24} = 2^8 = 256$ адресов: от 192.168.1.0 до 192.168.1.255.

Две подсети **пересекаются**, если существует хотя бы один IP-адрес, который принадлежит обеим подсетям одновременно.

Входные данные

Первая строка содержит целое число N ($2 \leq N \leq 10^5$) — количество подсетей в компании.

Следующие N строк содержат описания подсетей в формате: $a.b.c.d/m$, где:

- a, b, c, d — целые числа от 0 до 255
- m — целое число от 0 до 32

Выходные данные

Выведите YES, если среди данных подсетей есть хотя бы две пересекающиеся.

Выведите NO, если все подсети не пересекаются друг с другом.

Примечание

При прохождении всех тестов, ваше решение получит 100 баллов.

В примере 1 подсети 192.168.1.0/24 и 192.168.1.128/25 пересекаются (например, адрес 192.168.1.200 принадлежит обеим подсетям).

Решение

Python

```
def ip_to_int(ip):
    """Преобразует IP-адрес в целое число"""
    parts = ip.split('.')
    return (int(parts[0]) << 24) + (int(parts[1]) << 16) + (int(parts[2]) << 8) + int(parts[3])

def subnet_to_range(subnet):
    """Преобразует подсеть в диапазон [from, to)"""
    ip, mask = subnet.split('/')
    mask = int(mask)

    base_ip = ip_to_int(ip)

    network_start = (base_ip >> (32 - mask)) << (32 - mask)

    network_end = network_start + (1 << (32 - mask)) - 1

    return [network_start, network_end]

def check_subnets():
    """Проверяет пересечение подсетей"""
    n = int(input())

    ranges = []
    for _ in range(n):
        subnet = input().strip()
        ranges.append(subnet_to_range(subnet))

    ranges.sort()
```

```
for i in range(len(ranges) - 1):
    current_end = ranges[i][1]
    next_start = ranges[i + 1][0]

    if next_start <= current_end:
        print("YES")
        return

print("NO")

check_subnets()
```

Задача В

Условие: "TRON: Dice"

В секретном центре управления сорок второго энергоблока, построенного корпорацией «Р», специальный дроид-ассистент DICE-TRON проходит испытания в автоматическом управлении сложной системой распределительных каналов. Сеть каналов организована в виде огромного ориентированного дерева: из каждого узла расходятся трубы к другим модулям блока, и нигде пути назад не предусмотрено — радиация обязывает следовать только вперёд!

В каждом узле — от 0 до n труб (где n — число наименованных труб, оно фиксировано для всего лабиринта). DICE-TRON начинает путь из управляющего центра (узел 1).

На каждом шаге дроид бросает "энергетический" кубик на n граней, чтобы выбрать направление движения:

- Если в текущем узле k_i труб ($k_i \geq 1$), то дроид интерпретирует результат броска как число от 0 до $n - 1$ (dice). Дроид проходит в трубу, равную $(dice \bmod k_i)$ -й** по списку детей (нумерация с нуля).
- Если $k_i = 0$ (лист), то путешествие заканчивается.
- ВНИМАНИЕ: Если из узла выходит только одна труба, дроид всё равно бросает кубик и всегда переходит в единственную трубу.

Твоя задача — **вычислить математическое ожидание количества ходов** (переходов по трубам), необходимых для дроида, чтобы впервые попасть в лист из управляющего центра.

Входные данные

Первая строка: $n\ m$ — количество граней кубика ($2 \leq n \leq 20$), количество узлов в дереве ($2 \leq m \leq 10^5$).

Далее m строк: Каждая строка описывает i -й узел:

$k\ c_1\ c_2\ \dots\ c_k$

- k — количество детей узла ($0 \leq k \leq n$)
 - $c_0, \dots, c_{k-1}$ — номера детей ($2 \leq c_i \leq m$, если $k = 0$, строка оканчивается на k)
- Вершины нумеруются с 1. Управляющий центр — вершина 1.

Выходные данные

Выведи единственное вещественное число — математическое ожидание количества **ходов** (переходов по трубам), требуемое для достижения любого листа, начиная с управляющего центра.

Ответ засчитывается, если абсолютная или относительная ошибка не превышает 10^{-6} .

Примечание

Для примера 1 структура дерева:
вершина 1 ("управляющий центр") имеет двух детей: 2 и 7.

- $2 \rightarrow 3, 4$
 - $3 \rightarrow 8$
 - 8 — лист
 - $4 \rightarrow 5, 6$
 - 5 — лист
 - 6 — лист
- $7 \rightarrow 9$
 - 9 — лист

Мат ожидание кол-ва ходов:

- 5, 6, 8, 9 — **листы**: $E_5 = E_6 = E_8 = E_9 = 0$
- Узел 4: дети 5 и 6:
 $E_4 = 1 + \frac{E_5 + E_6}{2} = 1$
- Узел 3: один ребёнок 8:
 $E_3 = 1 + E_8 = 1$
- Узел 7: один ребёнок 9:
 $E_7 = 1 + E_9 = 1$
- Узел 2: дети 3 и 4:
 $E_2 = 1 + \frac{E_3 + E_4}{2} = 2$
- Узел 1 (корень): дети 2 и 7:
 $E_1 = 1 + \frac{E_2 + E_7}{2} = 2.5$

Решение

C++

```
#include <functional>
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

#define iom \
    cin.tie(0); \
    cout.tie(0); \
    ios_base::sync_with_stdio(false);
#define out cout.flush();
#define endl '\n'

using ld = long double;

void solve() {
    int n, m;
    cin >> n >> m;

    vector<vector<int>> children(m + 1);
    vector<ld> expected(m + 1, 0.0);

    for (int i = 1; i <= m; i++) {
        int k;
        cin >> k;
        children[i].resize(k);
        for (int j = 0; j < k; j++) {
            cin >> children[i][j];
        }
    }

    function<ld(int)> dfs = [&](int v) -> ld {
        if (children[v].empty()) {
            return expected[v] = 0.0;
        }

        int k = children[v].size();
        ld sum = 0.0;

        for (int i = 0; i < k; i++) {
            int child = children[v][i];
            ld child_expected = dfs(child);

            ld prob;
            if (i < n % k) {
                prob = (ld)(n / k + 1) / n;
            } else {
                prob = (ld)(n / k) / n;
            }

            sum += prob * child_expected;
        }

        return expected[v] = 1.0 + sum;
    };

    double result = dfs(1);
    cout << fixed << setprecision(10) << result << endl;
}

int32_t main() {
    iom;

    solve();

    out;
    return 0;
}
```

Python

```
from typing import List, Dict

def solve():
    n, m = map(int, input().split())

    children: List[List[int]] = [[] for _ in range(m + 1)]
    for i in range(1, m + 1):
        line = list(map(int, input().split()))
        k = line[0]
        if k > 0:
            children[i] = line[1:k+1]

    expected: Dict[int, float] = {}
```

```
def dfs(v: int) -> float:
    """Calculate expected number of moves from node v to any leaf"""
    if v in expected:
        return expected[v]

    if not children[v]:
        expected[v] = 0.0
        return 0.0

    k = len(children[v])
    sum_prob_expected = 0.0

    for i in range(k):
        child = children[v][i]
        child_expected = dfs(child)

        if i < n % k:
            prob = (n // k + 1) / n
        else:
            prob = (n // k) / n

        sum_prob_expected += prob * child_expected

    result = 1.0 + sum_prob_expected
    expected[v] = result
    return result

result = dfs(1)
print(f"{result:.10f}")

if __name__ == "__main__":
    solve()
```

Задача C

Условие: "Линейный ускоритель"

В подземных лабораториях научного центра "Р" завершилось строительство уникального линейного ускорителя. Тщательно нацеленный и очищенный канал длиной в сотни метров готов встретить свою главную героиню — заряжённую частицу, которой предстоит пройти серию контролируемых испытаний.

В момент времени $t = 0$ частица с зарядом q и массой m находится в начале пути ($x = 0$) — готовой к старту, её скорость $v_0 = 0$.

В ходе эксперимента сотрудники "Р" программируют ускоряющие секции: на контрольных участках линейного ускорителя периодически включаются и выключаются электрические поля, которые либо разгоняют, либо замедляют частицу (но никогда настолько, чтобы она пошла назад!). Всего задано T временных отрезков, в каждый из которых на траектории частицы действует заданное постоянное электрическое поле напряжённостью E_i (может быть как положительно, так и отрицательно направленным).

Вне указанных временных отрезков электрическое поле выключено ($E = 0$), и частица движется равномерно — инерция!

Теперь команда исследователей хочет узнать: когда окажется частица в избранных позициях, чтобы синхронизировать работу диагностических модулей? Для каждого заданной позиции X_j вычислите момент времени, в который частица будет находится там.

Входные данные

Первая строка: целое число T ($1 \leq T \leq 10^5$) — количество временных интервалов.
Далее T строк по три целых числа в каждой: $Start_i$, $Finish_i$ и E_i — момент начала действия, момент конца действия и напряжённость поля на этом интервале. $0 < Start_i < Finish_i \leq 10^9$, $|E_i| \leq 10^9$
Затем одна строка: два числа через пробел $q\ m$ — заряд и масса частицы ($0 < q \leq 10^5$, $0 < m \leq 10^5$).
Следующая строка: целое Q ($1 \leq Q \leq 10^5$) — число интересующих учёных позиций.
Далее Q строк по одному целому числу — X_j ($0 \leq X_j \leq 10^{18}$) — позиция очередного запроса.

Выходные данные

Для каждого из Q запросов выведите время t частицы в позиции X_j с точностью не менее 10^{-7} . Ответы выводите по порядку запросов.

Примечание

- Во все моменты времени скорость частицы неотрицательна, частица никогда не движется назад.
- Интервалы не накладываются друг на друга — в каждый момент может действовать максимум одно поле.
- Интервалы идут в порядке возрастания Finish. Гарантируется, что $Finish_{i-1} < Finish_i$.
- Запросы могут идти в любом порядке.

Для примера 1:

- 1** — число временных интервалов ($T=1$).
- 1 30 9** — с **1-й** по **30-ю** секунду (время), на частицу действует электрическое поле напряжённостью **9** (единиц).
- 4.565777 1.949875** — заряд $q=4.565777$, масса $m=1.949875$.
- 5** — интересующих позиций ($Q = 5$).
- Далее по одной в строке: позиции, в которые спрашивается, когда частица туда попадёт:
 - 597
 - 286

- 800
- 39
- 950

Как происходит движение частицы:

1. Начальная скорость в $t=0$ — $v_0=0$.
2. **На $[0, 1)$** и после **30** поля нет (ускорение 0): частица покоится или движется равномерно.
3. **На отрезке $[1, 30)$** секунда действует постоянное электрическое поле (обеспечивает постоянное ускорение).
4. (q, m) — позволяет вычислить ускорение: $a_i = \frac{q \cdot E_i}{m}$

Примерная траектория:

- Время 0 до 1: поле выключено, скорость 0, частица стоит на месте.
- Время 1 до 30: действует постоянное ускорение.
- После 30: частица летит равномерно с той скоростью, которую набрала к 30 секунде.

Для каждой позиции X_i :

1. Нужно определить, на каком отрезке (по времени) частица окажется в этой координате:
 - На отрезке покоя? На отрезке ускорения? На отрезке равномерного движения?
2. Найти соответствующее время—решить обратную задачу по формулам движения:
 - На отрезке ускорения: воспользуйтесь формулой $x = x_0 + v_0 t + \frac{1}{2} a t^2$.
 - На отрезке равномерного: $x = x_0 + v t$.

Решение

C++

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

using ll = long long;
using ld = long double;

struct TEvent {
    ld t_start;
    ld t_end;
    ld E;
};

struct TRange {
    ld X_start;
    ld V_start;
    ld Acceleration;
    ld Duration;
    ld TStart;
    bool Linear;
};

struct TQuery {
    ld Position;
    int Id;
    ld Responce;
};

ld SquareRootSolution(ld a, ld b, ld c, bool linear) {
    if (linear) {
        return -c / b;
    }
    ld d = b * b - 4 * a * c;
    if (a > 0) {
        return (-b + sqrt(d)) / (a + a);
    } else {
        return (-b - sqrt(d)) / (a + a);
    }
}

int main() {
    cin.tie(nullptr);
    cout.tie(nullptr);
    ios_base::sync_with_stdio(false);
    int t;
    cin >> t;
    std::vector<TEvent> events(t);
    for (auto &event : events) {
        cin >> event.t_start >> event.t_end >> event.E;
        assert(event.t_start < event.t_end);
    }
    for (int i = 1; i < t; ++i) {
        assert(events[i - 1].t_end < events[i].t_start);
    }
}
```

```

}

ld m, q;
cin >> q >> m;
std::vector<TRange> ranges;
ld cur_x = 0;
ld cur_v = 0;
ranges.push_back(TRange{
    .X_start = 0,
    .V_start = 0,
    .Acceleration = 0,
    .Duration = events[0].t_start,
    .TStart = 0,
    .Linear = true,
});
for (int i = 0; i < t; ++i) {
    ld acceleration = events[i].E * q / m;
    ld duration = events[i].t_end - events[i].t_start;
    ranges.push_back(TRange{.X_start = cur_x,
        .V_start = cur_v,
        .Acceleration = acceleration,
        .Duration = duration,
        .TStart = events[i].t_start,
        .Linear = false});
    cur_x += cur_v * duration + acceleration * duration * duration / (ld)2.0;
    cur_v += acceleration * duration;
    assert(cur_v > 0);
    if (cur_x > 1e18) {
        break;
    }
    if (i + 1 != t) {
        ranges.push_back(
            TRange{.X_start = cur_x,
                .V_start = cur_v,
                .Acceleration = 0,
                .Duration = events[i + 1].t_start - events[i].t_end,
                .TStart = events[i].t_end,
                .Linear = true});
        cur_x += cur_v * (events[i + 1].t_start - events[i].t_end);
    }
    assert(cur_v > 0);
    if (cur_x > 1e18) {
        break;
    }
}

if (!ranges.empty() && cur_v > 0) {
    ranges.push_back(TRange{.X_start = cur_x,
        .V_start = cur_v,
        .Acceleration = 0,
        .Duration = 1e18,
        .TStart = events[t - 1].t_end,
        .Linear = true});
}

int query_count;
cin >> query_count;
std::vector<TQuery> queries(query_count);
for (int i = 0; i < query_count; ++i) {
    cin >> queries[i].Position;
    queries[i].Id = i;
}

sort(queries.begin(), queries.end(),
    [](const TQuery &lhs, const TQuery &rhs) {
        return lhs.Position < rhs.Position;
    });
int uk_in_ranges = 0;
for (auto &query : queries) {
    if (query.Position == 0) {
        query.Responce = 0;
        continue;
    }

    while (uk_in_ranges + 1 < ranges.size() &&
        ranges[uk_in_ranges + 1].X_start <= query.Position) {
        ++uk_in_ranges;
    }

    ld dx = ranges[uk_in_ranges].X_start - query.Position;
    query.Responce =
        ranges[uk_in_ranges].TStart +
        SquareRootSolution(ranges[uk_in_ranges].Acceleration / (ld)2.0,
            ranges[uk_in_ranges].V_start, dx,
            ranges[uk_in_ranges].Linear);
}
sort(queries.begin(), queries.end(),
    [](const TQuery &lhs, const TQuery &rhs) { return lhs.Id < rhs.Id; });
cout.precision(8);
for (auto &query : queries) {
    cout << fixed << query.Responce << '\n';
}
return 0;
}

```

Python

```
import math
import sys

class Segment:
    def __init__(self, start_time=0, end_time=0, acceleration=0, start_pos=0, start_vel=0):
        self.start_time = start_time
        self.end_time = end_time
        self.acceleration = acceleration
        self.start_pos = start_pos
        self.start_vel = start_vel

        if start_time != 0 or end_time != 0:
            duration = end_time - start_time
            self.end_pos = (start_pos + start_vel * duration +
                           0.5 * acceleration * duration * duration)
        else:
            self.end_pos = 0

    def get_position(self, t):
        dt = t - self.start_time
        return self.start_pos + self.start_vel * dt + 0.5 * self.acceleration * dt * dt

    def get_velocity(self, t):
        dt = t - self.start_time
        return self.start_vel + self.acceleration * dt

    def find_time_for_position(self, target_pos):
        if target_pos < self.start_pos:
            return -1

        target_dist = target_pos - self.start_pos

        if abs(self.acceleration) < 1e-12:
            if abs(self.start_vel) < 1e-12:
                return self.start_time if abs(target_dist) < 1e-12 else -1

            t = target_dist / self.start_vel
            if t < 0:
                return -1
            return self.start_time + t
        else:
            discriminant = self.start_vel * self.start_vel + 2 * self.acceleration * target_dist
            if discriminant < 0:
                return -1

            t1 = (-self.start_vel + math.sqrt(discriminant)) / self.acceleration
            t2 = (-self.start_vel - math.sqrt(discriminant)) / self.acceleration

            result = -1
            if t1 >= 0 and t1 <= self.end_time - self.start_time:
                result = self.start_time + t1
            if (t2 >= 0 and t2 <= self.end_time - self.start_time and
                (result < 0 or self.start_time + t2 < result)):
                result = self.start_time + t2

            return result

    def bin_search_by_end_pos(segments, target_pos):
        l = -1
        r = len(segments)
        while r - l > 1:
            mid = (l + r) // 2
            if segments[mid].end_pos < target_pos:
                l = mid
            else:
                r = mid
        return r

    def solve():
        T = int(input())

        intervals = []
        for _ in range(T):
            start_t, end_t, field = map(int, input().split())
            intervals.append((start_t, end_t, field))

        q, m = map(float, input().split())

        segments = []
        current_pos = 0.0
        current_vel = 0.0
        last_end_time = 0

        for start_t, end_t, field in intervals:
            if start_t > last_end_time and current_vel > 0:
                segments.append(Segment(last_end_time, start_t, 0, current_pos, current_vel))

            gap_duration = start_t - last_end_time
            current_pos += current_vel * gap_duration

            acceleration = q * field / m
```

```
segments.append(Segment(start_t, end_t, acceleration, current_pos, current_vel))

duration = end_t - start_t
current_pos += current_vel * duration + 0.5 * acceleration * duration * duration
current_vel += acceleration * duration

assert current_vel >= 0

last_end_time = end_t

# Add infinite segment for uniform motion after last interval
if segments and current_vel > 0:
    segments.append(Segment(intervals[-1][1], int(1e18), 0, current_pos, current_vel))

Q = int(input())

for _ in range(Q):
    X = float(input())

    if X == 0:
        print("0")
        continue

    idx = bin_search_by_end_pos(segments, X)

    result = -1
    for i in range(max(0, idx - 1), min(len(segments), idx + 2)):
        t = segments[i].find_time_for_position(X)
        if (t >= 0 and t >= segments[i].start_time and
            (segments[i].end_time >= 1e17 or t <= segments[i].end_time)):
            if result < 0 or t < result:
                result = t

    assert result != -1
    print(f"{result:.10f}")

if __name__ == "__main__":
    solve()
```

Задача D

Условие: "Силовое поле"

На далёкой планете расположена научная база землян. Для защиты от метеоритных дождей вокруг базы установлены N генераторов силового поля. Каждый генератор находится в точке с целочисленными координатами (x, y) на поверхности планеты.

Генераторы создают защитный купол, форма которого при взгляде сверху представляет собой выпуклый многоугольник — каждый генератор соединён с двумя соседними, образуя непрерывный контур защиты.

Инженеры базы постоянно оптимизируют систему защиты. Время от времени они перемещают генераторы на новые позиции, чтобы улучшить покрытие.

При этом соблюдаются важные условия:

- Контур защиты остаётся выпуклым многоугольником
- Генератор может поменяться местами только с одним из своих непосредственных соседей или остаться между теми же соседями

Команда инженеров хочет знать новую площадь защищенной территории после каждого перемещения, поэтому просит вас помочь её вычислить.

Входные данные

В первой строке заданы два целых числа N и Q ($3 \leq N \leq 10^5, 1 \leq Q \leq 10^5$) — количество генераторов и количество операций перемещения.

Следующие N строк содержат по два целых числа x_i и y_i ($-10^9 \leq x_i, y_i \leq 10^9$) — координаты i -го генератора ($1 \leq i \leq N$). Гарантируется, что все генераторы образуют выпуклый многоугольник.

Следующие Q строк содержат по три целых числа i, x, y ($1 \leq i \leq N, -10^9 \leq x, y \leq 10^9$) — номер перемещаемого генератора и его новые координаты.

Выходные данные

Выведите Q чисел — площадь защищаемой территории (с точностью до двух знаков после запятой) после каждой операции.

Примечание

В этой задаче баллы выставляются за группы тестов. Каждая группа будет тестироваться только при успешном тестировании предыдущей.

Группа	Баллы	Ограничения
1	30	$N, Q \leq 500$
2	30	$N, Q \leq 10^5$, генераторы после перемещении остаются между теми же генераторами в контуре, что и до перемещения

Группа	Баллы	Ограничения
3	40	$N, Q \leq 10^5$

До начала перемещений защитный контур в порядке обхода по часовой стрелке равен [1, 4, 2, 3]

После первого перемещения порядок остаётся тот же.

После второго перемещения порядок равен [1, 2, 4, 3].

После третьего перемещения порядок равен [2, 1, 4, 3].

Решение

C++

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <map>
#include <numeric>
#include <ostream>
#include <print>
#include <random>
#include <ranges>
#include <set>
#include <stack>
#include <unordered_map>
#include <unordered_set>
#include <vector>

#define sz(a) ((int)((a).size()))
#define make_unique(x) \
    sort((x).begin(), (x).end()); \
    (x).erase(unique((x).begin(), (x).end()), (x).end())

using namespace std;
// mt19937 rnd(239);
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

typedef long long ll;
typedef long double ld;

struct Point {
    ll x;
    ll y;
};

ll cross_product(const Point& o, const Point& a, const Point& b) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

ll calc_area(vector<Point>& pts, vector<int>& order) {
    ll area = 0;
    int n = pts.size();
    Point zero = Point{0, 0};
    for (int i = 0; i < n; ++i) {
        area += cross_product(zero, pts[order[i]], pts[order[(i + 1) % n]]);
    }
    return area;
}

bool check_convex(const vector<Point>& pts, const vector<int>& order, int pos) {
    int n = pts.size();
    int prev = (pos - 1 + n) % n;
    int pprev = (prev - 1 + n) % n;
    int next = (pos + 1) % n;
    int nnext = (next + 1) % n;
    auto cp1 = cross_product(pts[order[pprev]], pts[order[prev]], pts[order[pos]]);
    auto cp2 = cross_product(pts[order[prev]], pts[order[pos]], pts[order[next]]);
    auto cp3 = cross_product(pts[order[pos]], pts[order[next]], pts[order[nnext]]);
    return cp1 > 0 && cp2 > 0 && cp3 > 0;
}

void print_ans(ll area) {
    area = (abs(area));
    if (area % 2 == 0) {
        cout << area / 2 << ".00\n";
    } else {
        cout << area / 2 << ".50\n";
    }
}

void solve() {
    int n, q;
    cin >> n >> q;
    vector<Point> pts(n);
    for (int i = 0; i < n; ++i) {
        cin >> pts[i].x >> pts[i].y;
    }
}
```

```

vector<int> order(n);
std::iota(begin(order), end(order), 0);
{
    ld c_x = [&pts]() -> ld {
        ll sm = 0;
        for (auto& pt : pts) {
            sm += pt.x;
        }
        return static_cast<ld>(sm) / pts.size();
    }();
    ld c_y = [&pts]() -> ld {
        ll sm = 0;
        for (auto& pt : pts) {
            sm += pt.y;
        }
        return static_cast<ld>(sm) / pts.size();
    }();

    ranges::sort(order, [&c_x, c_y, pts](const int& l, const int& r) -> bool {
        return atan2(static_cast<ld>(pts[l].y) - c_y, static_cast<ld>(pts[l].x) - c_x) <
            atan2(static_cast<ld>(pts[r].y) - c_y, static_cast<ld>(pts[r].x) - c_x);
    });
}

vector<int> positions(n);
for (int i = 0; i < n; ++i) {
    positions[order[i]] = i;
}

ll area = calc_area(pts, order);
cout << setprecision(2);
cout << fixed;
Point zero = Point{0, 0};
while (q--) {
    int idx;
    ll nx, ny;
    cin >> idx >> nx >> ny;
    --idx;
    int pos = positions[idx];
    int prev = (pos - 1 + n) % n;
    int next = (pos + 1) % n;
    area -= cross_product(zero, pts[order[prev]], pts[order[pos]]);
    area -= cross_product(zero, pts[order[pos]], pts[order[next]]);
    pts[idx].x = nx;
    pts[idx].y = ny;
    if (check_convex(pts, order, pos)) {
        area += cross_product(zero, pts[order[prev]], pts[order[pos]]);
        area += cross_product(zero, pts[order[pos]], pts[order[next]]);
        print_ans(area);
        continue;
    }

    area += cross_product(zero, pts[order[prev]], pts[order[next]]);
    swap(order[pos], order[prev]);
    if (check_convex(pts, order, prev)) {
        swap(positions[order[pos]], positions[order[prev]]);
        int pprev = (prev - 1 + n) % n;
        area -= cross_product(zero, pts[order[pprev]], pts[order[pos]]);
        area += cross_product(zero, pts[order[pprev]], pts[order[prev]]);
        area += cross_product(zero, pts[order[prev]], pts[order[pos]]);
        print_ans(area);
        continue;
    }
    swap(order[pos], order[prev]);
    swap(order[pos], order[next]);
    swap(positions[order[pos]], positions[order[next]]);
    int nnext = (next + 1) % n;
    area -= cross_product(zero, pts[order[pos]], pts[order[nnext]]);
    area += cross_product(zero, pts[order[pos]], pts[order[next]]);
    area += cross_product(zero, pts[order[next]], pts[order[nnext]]);
    print_ans(area);
    continue;
}
}

int32_t main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    while (t--) {
        solve();
    }
    return 0;
}

```

Python

```

import math
import sys
from typing import List, Tuple

```

```

def sz(a):
    return len(a)

class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

def cross_product(o: Point, a: Point, b: Point) -> int:
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x)

def calc_area(pts: List[Point], order: List[int]) -> int:
    area = 0
    n = len(pts)
    zero = Point(0, 0)
    for i in range(n):
        area += cross_product(zero, pts[order[i]], pts[order[(i + 1) % n]])
    return area

def check_convex(pts: List[Point], order: List[int], pos: int) -> bool:
    n = len(pts)
    prev = (pos - 1 + n) % n
    pprev = (prev - 1 + n) % n
    next = (pos + 1) % n
    nnext = (next + 1) % n

    cp1 = cross_product(pts[order[pprev]], pts[order[prev]], pts[order[pos]])
    cp2 = cross_product(pts[order[prev]], pts[order[pos]], pts[order[next]])
    cp3 = cross_product(pts[order[pos]], pts[order[next]], pts[order[nnext]])

    return cp1 > 0 and cp2 > 0 and cp3 > 0

def print_ans(area: int):
    area = abs(area)
    if area % 2 == 0:
        print(f"{area // 2}.00")
    else:
        print(f"{area // 2}.50")

def solve():
    n, q = map(int, input().split())
    pts = []
    for _ in range(n):
        x, y = map(int, input().split())
        pts.append(Point(x, y))

    order = list(range(n))

    c_x = sum(pt.x for pt in pts) / len(pts)
    c_y = sum(pt.y for pt in pts) / len(pts)

    order.sort(key=lambda i: math.atan2(pts[i].y - c_y, pts[i].x - c_x))

    positions = [0] * n
    for i in range(n):
        positions[order[i]] = i

    area = calc_area(pts, order)
    zero = Point(0, 0)

    for _ in range(q):
        idx, nx, ny = map(int, input().split())
        idx -= 1

        pos = positions[idx]
        prev = (pos - 1 + n) % n
        next = (pos + 1) % n

        area -= cross_product(zero, pts[order[prev]], pts[order[pos]])
        area -= cross_product(zero, pts[order[pos]], pts[order[next]])

        pts[idx].x = nx
        pts[idx].y = ny

        if check_convex(pts, order, pos):
            area += cross_product(zero, pts[order[prev]], pts[order[pos]])
            area += cross_product(zero, pts[order[pos]], pts[order[next]])
            print_ans(area)
            continue

        area += cross_product(zero, pts[order[prev]], pts[order[next]])
        order[pos], order[prev] = order[prev], order[pos]

        if check_convex(pts, order, prev):
            positions[order[pos]], positions[order[prev]] = (
                positions[order[prev]],
                positions[order[pos]],
            )

```

```
pprev = (prev - 1 + n % n
area -= cross_product(zero, pts[order[pprev]], pts[order[pos]])
area += cross_product(zero, pts[order[pprev]], pts[order[prev]])
area += cross_product(zero, pts[order[prev]], pts[order[pos]])
print_ans(area)
continue

order[pos], order[prev] = order[prev], order[pos]
order[pos], order[next] = order[next], order[pos]

positions[order[pos]], positions[order[next]] = (
    positions[order[next]],
    positions[order[pos]],
)
nnext = (next + 1) % n
area -= cross_product(zero, pts[order[pos]], pts[order[nnext]])
area += cross_product(zero, pts[order[pos]], pts[order[next]])
area += cross_product(zero, pts[order[next]], pts[order[nnext]])
print_ans(area)

def main():
    solve()

if __name__ == "__main__":
    main()
```

Задача Е

Условие: "Охота на вирус"

В недалёком цифровом будущем сеть ядерных реакторов соединена надёжными туннелями передачи данных — они образуют дерево из N цифровых узлов. Один из узлов оказывается заражён — в нём активируется вирус. В заранее заданной вершине номер 7 находитесь Вы — цифровой агент безопасности, задача которого состоит в том, чтобы как можно быстрее выявить и нейтрализовать вирус, не допустив его распространения.

Первый ход за Вами - может остаться на месте или перейти по одному из туннелей в соседний узел. Затем ход делает вирус, также — либо остаться, либо перейти в соседний узел. Далее ходы повторяются поочерёдно: сначала Вы, потом вирус, и так далее. Если в какой-то момент вирус и агент оказываются в одном узле после чьего-либо хода, считается, что вирус нейтрализован. С этого момента заражение остановлено.

Будем измерять длительность заражения в количестве ваших ходов до момента его поимки. То есть, если агент сделал один ход, вирус — свой, затем после вашего хода вирус оказался в том же узле, что и вы, то длительность заражения будет равна 2 (два хода агента).

Вирус действует стремясь выжить как можно дольше, агент — стремится уничтожить вирус как можно быстрее.

Пусть $t(u)$ — время жизни вируса, если он стартует в узле u . Ваша задача - вычислить единственное число, равное сумме $t(u)$ по всем вершинам дерева, т. е. вывести: $\sum_{u=1}^N t(u)$

Входные данные

В первой строке вводится целое число N ($7 \leq N \leq 4000000$) - количество вершин в дереве.
В следующих $(N - 1)$ строках вводится по 2 целых числа a_i и b_i через пробел ($1 \leq a_i, b_i \leq N, a_i \neq b_i$) - концы очередного ребра.
Гарантируется, что вводимый граф - дерево.

Выходные данные

Выведите единственное число - ответ на задачу.

Примечание

Группа	Баллы	Ограничения
1	4	$(N \leq 90)$
2	7	$(N \leq 900)$
3	23	$(N \leq 9000)$
4	33	$(N \leq 100000)$
5	33	$(N \leq 4000000)$

Необходимо

- 0 ходов для вершины 7; агент сразу ловит вирус
 - 1 ход для вершины 10; агент делает 1 ход и ловит вирус
 - 5 ходов для вершин 1, 2, 8, 9; вирус бежит в вершину 9
 - 7 ходов для вершин 3, 4, 5, 6, 11, 12; вирус бежит в вершину 12
- Итого $1 * 1 + 5 * 4 + 7 * 6 = 1 + 20 + 42 = 63$

Решение

C++

```
#include <iostream>
#include <cassert>
#include <vector>

using namespace std;

const int MAXN = 1000000;
const int AGENT_NODE = 6;

vector<int> gr[MAXN];
int best_son[MAXN];
long long ans = 0;

void dfs(int v, int p, int dist) {
    best_son[v] = dist;
    for (auto to : gr[v]) {
        if (to != p) {
            dfs(to, v, dist + 1);
            best_son[v] = max(best_son[v], best_son[to]);
        }
    }
}

vector<int> path;
void dfs_ans(int v) {
    path.push_back(v);
    // check text task solutions to understand this comment.
    // edges = path.size() - 1
    // if enemy makes k moves, agent makes (k+2) moves.
    // so node reachable if k + (k + 2) < path.size() - 1
    // so k < (path.size() - 3) / 2. k = (path.size() - 3) / 2
    // best node has (path.size() - 1 - k) index
    if (path.size() == 2) {
        ++ans;
    } else if (path.size() >= 3) {
        int k = (path.size() - 3) / 2;
        int best_node = path.size() - 1 - k;
        ans += best_son[path[best_node]];
    }
    for (auto to : gr[v]) {
        if (path.size() < 2 || to != path[path.size() - 2]) {
            dfs_ans(to);
        }
    }
    path.pop_back();
}

void solve() {
    int n;
    cin >> n;
    for (int i = 1; i < n; ++i) {
        int a, b;
        cin >> a >> b;
        --a;
        --b;
        gr[a].push_back(b);
        gr[b].push_back(a);
    }
    dfs(AGENT_NODE, AGENT_NODE, 0);
    dfs_ans(AGENT_NODE);
    cout << ans << endl;
}

int main() {
    cin.tie(nullptr);
    cout.tie(nullptr);
    ios_base::sync_with_stdio(false);
    int t = 1;
    while (t--) {
        solve();
    }
    return 0;
}
```

Python

```
import sys

AGENT_NODE = 6 # 0-based index, 7-й узел (если считать с 1)

def dfs_iterative(start, gr, best_son):
    stack = [(start, -1, 0)]
    parent = [None] * len(gr)
    depth = [0] * len(gr)
    visited = [False] * len(gr)
    order = []
```

```

# First pass: simulate entering nodes
while stack:
    v, p, d = stack.pop()
    if visited[v]:
        continue
    visited[v] = True
    parent[v] = p
    depth[v] = d
    order.append(v)
    for to in gr[v]:
        if to != p:
            stack.append((to, v, d + 1))

# Second pass: simulate post-order processing
for v in reversed(order):
    best_son[v] = depth[v]
    for to in gr[v]:
        if to != parent[v]:
            best_son[v] = max(best_son[v], best_son[to])

def dfs_ans_iterative(start, gr, best_son):
    ans = [0]
    path = []
    stack = [(start, False, None)] # (node, is_backtrack, parent)

    while stack:
        v, is_backtrack, p = stack.pop()

        if is_backtrack:
            path.pop()
            continue

        path.append(v)
        stack.append((v, True, p)) # mark node for removal after children

        if len(path) == 2:
            ans[0] += 1
        elif len(path) >= 3:
            k = (len(path) - 3) // 2
            best_node = len(path) - 1 - k
            ans[0] += best_son[path[best_node]]

        for to in gr[v]:
            if to != p:
                stack.append((to, False, v))

    return ans[0]

def main():
    input = sys.stdin.readline
    n = int(input())
    gr = [[] for _ in range(n)]
    for _ in range(n - 1):
        a, b = map(int, input().split())
        a -= 1
        b -= 1
        gr[a].append(b)
        gr[b].append(a)

    best_son = [0] * n
    dfs_iterative(AGENT_NODE, gr, best_son)
    result = dfs_ans_iterative(AGENT_NODE, gr, best_son)
    print(result)

if __name__ == "__main__":
    main()

```